Plan Maestro de Pruebas — Huellitas Saludables PRO

Versión: 1.0
Fecha de revisión: 13 de noviembre de 2026
Estado: plan documental; las pruebas pendientes requieren planificación y ejecución.
Proyecto: plataforma web para gestión de clínica veterinaria.

> Este plan toma como referencia la estructura del ejemplo de Plan Maestro de Pruebas adjunto. Describe tanto las pruebas que se observan en el repositorio como las brechas. La palabra «implementada» indica que existe código o evidencia documental; no significa que se haya vuelto a ejecutar durante la elaboración de este plan.

1. Introducción

1.1 Objetivo

Establecer el alcance, estrategia, entorno, responsabilidades y criterios de evaluación para verificar Huellitas Saludables PRO. El plan separa la cobertura automatizada actual del backend, las verificaciones manuales documentadas y las pruebas que todavía faltan, especialmente frontend/E2E y pruebas no funcionales.

Las categorías de prueba se organizan en cuatro dimensiones independientes: forma de ejecución, nivel, conocimiento del sistema y propósito. No son mutuamente excluyentes: por ejemplo, una prueba automatizada de regresión puede ser de integración, caja negra respecto al endpoint y caja gris respecto a la base de datos.

1.2 Documentos y fuentes relacionados

- Guía de ejecución de pruebas.
- Índice de documentación de pruebas.
- Documentación del frontend.
- Guía de arranque.
- `back/package.json` y `back/vitest.config.js`.
- Suites actuales: `back/tests/unit/` y `back/tests/integration/`.
- `BITACORA_VALIDACION_FORMULARIOS.md`, evidencia de una ejecución anterior.

- `ARQUITECTURA_REGLAS_Y_DECISIONES_TECNICAS.md`, reglas de negocio y estado técnico.

2. Alcance de las pruebas

2.1 Resumen de alcance y estado

Área	Alcance previsto	Estado observado
Backend: servicios, validaciones, seguridad y utilidades	Unitarias y pruebas de integración HTTP con PostgreSQL	**Implementado** en el código de pruebas. Hay un resultado histórico de 23 archivos y 334 pruebas aprobadas registrado el 13/11/2026; no se reejecutó al redactar este plan.
Frontend: React, formularios, rutas y estados visuales	Verificación manual y futura automatización de componentes/E2E	**Parcial/manual.** Se documentaron comprobaciones manuales de formularios. No se encontró suite automatizada de frontend ni E2E.
Sistema completo	Browser/Nginx → API → PostgreSQL y flujos de usuario	**Pendiente como suite formal.** El healthcheck de API/BD no equivale a una prueba de sistema de extremo a extremo.
Aceptación	Validación de flujos y criterios por representante del negocio	**Pendiente formal.** No se encontró acta de aceptación ni matriz aprobada por usuarios finales.
Seguridad	Validación automatizada focalizada en backend	**Parcial.** Existen pruebas de JWT, contraseñas, validación y SQL Injection; falta una evaluación integral/penetración documentada.
Rendimiento, carga y estrés	Medición en entorno representativo	**Pendiente.** No se encontró herramienta, benchmark, perfil de carga ni umbral acordado.
Usabilidad y compatibilidad	Evaluación con usuarios y matriz de navegadores/dispositivos	**Parcial / pendiente.** Hay observaciones manuales de interfaz, no un protocolo formal ni pruebas cruzadas registradas.

2.2 Incluido en este plan

Se incluyen registro e inicio de sesión; usuarios y roles; mascotas; disponibilidad de especialistas; reservas, cancelaciones y vencimiento automático de citas; historial clínico; validación de formularios; persistencia y respuestas API; y aspectos de seguridad, usabilidad, compatibilidad y desempeño que deben planificarse.

Las pruebas deben proteger las reglas documentadas, entre ellas: permisos verificados en backend, teléfono numérico, correo/contraseña sin espacios, reserva en horario futuro con zona `America/Bogota`, prevención de reservas duplicadas y conservación/auditoría de cancelaciones.

2.3 Fuera de una primera corrida certificadora

Rendimiento, carga y estrés se consideran **pendientes de definición**, no aprobados ni excluidos permanentemente. Antes de ejecutarlos se deben acordar ambiente, datos sintéticos, concurrencia, duración, métricas y umbrales con el responsable del producto. Las pruebas dirigidas por IA no son un criterio autónomo de aprobación.

3. Clasificación de pruebas y brechas

3.1 Según la forma de ejecución

N.º	Tipo	De qué trata	Estado en Huellitas Saludables PRO y brecha
1	**Manual**	Una persona sigue pasos, observa resultados y registra evidencia sin que un runner automatice el caso completo. Útil para exploración visual, usabilidad y aceptación.	**Parcial: hay evidencia histórica.** La bitácora de formularios registra pruebas manuales de registro, login, errores en columna, restricciones del teléfono y mensajes del backend. Falta un protocolo repetible que cubra todos los roles y módulos.
2	**Automatizada**	Un runner ejecuta pasos y compara resultados con aserciones reproducibles. Reduce regresiones repetitivas.	**Implementada en backend.** Vitest y Supertest cubren unitarias e integración con API/BD. No se encontró runner de pruebas frontend ni E2E de navegador.
3	**Dirigida por IA**	IA puede ayudar a proponer casos, datos sintéticos, particiones o analizar resultados; las aserciones y decisiones de aprobación deben seguir siendo verificables y aprobadas por el equipo.	**No implementada como estrategia formal.** No se encontró pipeline/agente de IA que ejecute y califique pruebas. Si se incorpora, no enviar datos personales/producción y no aceptar respuestas generadas como resultado de prueba sin un oráculo determinista y revisión humana.

3.2 Según el nivel de la prueba

N.º	Tipo	De qué trata	Estado y trabajo faltante
4	**Unitaria**	Comprueba una función, servicio o unidad pequeña de forma aislada, con dependencias controladas cuando corresponda.	**Implementada en backend.** Hay suites para utilidades, middleware, servicios y modelos. Mantener pruebas deterministas y completar huecos de reglas de negocio.
5	**Integración**	Comprueba la interacción de componentes reales, por ejemplo HTTP → middleware → servicio/modelo → PostgreSQL.	**Implementada en backend.** `tests/integration/` usa Supertest y setup de base de datos de prueba. Mantener ejecución únicamente contra una base desechable.
6	**Sistema**	Verifica el producto integrado de punta a punta, con interfaz, API, base de datos y configuración de despliegue.	**Pendiente como suite completa.** Crear un flujo con navegador: login, mascota, reserva futura, atención/historia clínica y permisos; validar también rutas con Nginx/Compose. El healthcheck actual solo confirma salud de API/BD.
7	**Aceptación**	Confirma con criterios de negocio que el sistema permite completar las tareas acordadas y que el resultado es aceptable para usuarios responsables.	**Pendiente formal.** Definir escenarios y criterios con representante de clínica/usuario; registrar versión, datos, resultado, defectos y aprobación. Las verificaciones manuales previas no sustituyen un acta de aceptación.

3.3 Según el conocimiento del sistema

N.º	Tipo	De qué trata	Estado y trabajo faltante
8	**Caja blanca**	Diseña casos usando conocimiento de implementación, ramas, validaciones, consultas o código interno.	**Parcial/implementada en backend.** Las pruebas unitarias conocen módulos internos y existe una comprobación de seguridad que inspecciona patrones de SQL. No hay clasificación explícita por técnica ni cobertura mínima configurada.
9	**Caja negra**	Evalúa entradas y salidas por la interfaz o contrato observable, sin depender de detalles internos.	**Parcial.** Las pruebas HTTP comprueban rutas, status y cuerpos; algunas pruebas manuales cubren formularios. Falta una batería formal de flujos E2E basada en requisitos y roles.
10	**Caja gris**	Combina observación externa con conocimiento parcial de arquitectura, roles, esquema o datos de prueba.	**Parcial/inferida.** Las pruebas de integración acceden a la API con conocimiento de roles y usan una BD sembrada/resettable. Documentar explícitamente qué información interna utiliza cada caso y evitar acoplamiento innecesario al esquema.

3.4 Según el propósito: pruebas funcionales

N.º	Tipo	De qué trata	Estado y trabajo faltante
11	**Humo**	Conjunto pequeño tras un build/despliegue para confirmar que los caminos esenciales están disponibles y no hay una falla bloqueante.	**Parcial.** Hay prueba de `/health` y pruebas básicas de API. Falta definir y ejecutar un smoke de frontend + login + endpoint protegido en el artefacto desplegado.
12	**Regresión**	Repite casos ya aprobados tras cambios para detectar que una funcionalidad existente se rompió.	**Parcial.** La suite automatizada puede servir como regresión al ejecutarse en cada cambio, pero no se encontró pipeline CI ni colección etiquetada como regresión. Añadir matriz de regla→caso y política de ejecución por pull request/release.

3.5 Según el propósito: pruebas no funcionales

N.º	Tipo	De qué trata	Estado y trabajo faltante
13	**Rendimiento**	Mide latencia, throughput, uso de CPU/memoria y estabilidad bajo un perfil definido, normalmente con usuario/concurrencia controlados.	**Pendiente.** No hay benchmark ni umbrales aprobados. Acordar percentiles, transacciones representativas y topología antes de medir.
14	**Carga**	Mide comportamiento bajo la carga esperada o un perfil objetivo (usuarios/rutas/ritmo/duración).	**Pendiente.** No se encontraron pruebas/herramientas de carga. Definir perfil de citas, login y consultas; usar ambiente aislado y datos sintéticos.
15	**Estrés**	Lleva el sistema más allá de la carga prevista para identificar punto de quiebre, recuperación, errores y pérdida de datos.	**Pendiente.** No ejecutar contra producción. Definir límites, parada segura y recuperación; comprobar que concurrencia no permita reservas incompatibles.
16	**Usabilidad**	Observa si usuarios representativos pueden completar tareas de forma comprensible, accesible y con errores recuperables.	**Parcial en exploración, pendiente formal.** Se probó visualmente la presentación de errores y restricciones de formularios. Faltan protocolo/tareas, participantes, criterios, teclado/lectores de pantalla y resultados documentados.
17	**Seguridad**	Busca fallas de autenticación/autorización, exposición de datos, validación, inyección, gestión de secretos y otras amenazas.	**Parcial.** Hay pruebas automatizadas de JWT, passwords/bcrypt, SQL Injection y validación. Faltan pruebas más amplias de

N.º	Tipo	De qué trata	Estado y trabajo faltante
			permisos por rol/recurso, dependencias, configuración, sesiones/rate limit y evaluación independiente. No equivale a certificación o pentest.
18	**Compatibilidad**	Comprueba funcionamiento equivalente en navegadores, sistemas, resoluciones y dispositivos acordados.	**Pendiente.** No se encontró matriz ni resultados de navegadores/dispositivos. Definir combinaciones soportadas y probar responsive, formularios, sesión, navegación y accesibilidad básica.

4. Casos y escenarios incluidos

ID propuesto	Escenario	Tipos que puede cubrir	Estado inicial
TC-01	Registro: campos obligatorios, teléfono solo numérico, correo y contraseña sin espacios, respuesta por campo	Manual, automatizada, caja negra, funcional, usabilidad	Hay validaciones automatizadas backend y comprobación manual documentada; ampliar a frontend automatizado y regresión.
TC-02	Login correcto/incorrecto, JWT ausente, expirado o inválido; respuesta genérica a errores internos	Unitaria, integración, caja negra/blanca, seguridad	Parcialmente cubierto por suites backend existentes; verificar trazabilidad de los casos.
TC-03	Acceso a endpoint por rol permitido y denegado; usuario intenta consultar/modificar recurso ajeno	Integración, sistema, caja negra/gris, seguridad	Ampliar matriz rol×endpoint×propiedad del recurso. No confiar solo en guards del frontend.
TC-04	Crear mascota y consultar su relación con usuario	Integración, sistema, regresión	Cubierto en parte por pruebas de API; verificar casos de acceso cruzado y límites de datos.
TC-05	Crear disponibilidad futura y rechazar fecha pasada; zona `America/Bogota`	Unitaria, integración, caja blanca/negra, regresión	Hay utilidades y pruebas de backend; mantener reloj controlado y cubrir límites de fecha/zona.
TC-06	Dos solicitudes concurrentes para la misma franja: solo una reserva activa puede completarse	Integración, sistema, caja gris, seguridad/integridad	Proponer pruebas de concurrencia con PostgreSQL real; no confundir con prueba de carga.

ID propuesto	Escenario	Tipos que puede cubrir	Estado inicial
TC-07	Cancelación manual/automática: estado, motivo y única auditoría; cita y franja conservadas	Integración, sistema, caja gris, regresión	Hay pruebas backend de cancelación automática; cubrir idempotencia, carrera y conservación histórica.
TC-08	Especialista consulta cita autorizada y registra/actualiza historia clínica	Integración, sistema, aceptación, seguridad	Hay pruebas de historia/especialista; validar flujo completo desde UI y datos indebidos.
TC-09	Build frontend, rutas SPA y proxy Nginx/API	Smoke, sistema, compatibilidad	No hay E2E frontend constatado; proponer prueba automatizada tras build en Compose.
TC-10	Carga y degradación progresiva sobre login, consulta de citas y reserva	Rendimiento, carga, estrés	Pendiente hasta acordar umbrales, perfiles, ambiente y herramienta.

Los IDs TC-01–TC-10 son una propuesta de planificación; no representan diez casos ya implementados ni aprobados.

5. Entorno y configuración

- **Backend de prueba:** Node.js según `back/Dockerfile`, Express, Vitest y Supertest.
- **Base de datos:** PostgreSQL 16. Para el comando de Docker, `docker-compose.test.yml` crea el servicio `db-test` con almacenamiento temporal (`tmpfs`) y fija `DB_NAME_TEST=proyectohs_test`.
- **Frontend:** React/TypeScript/Vite. Para futura prueba de sistema se deberá incluir navegador automatizado y proxy Nginx/Compose cuando el objetivo sea verificar integración desplegada.
- **Datos:** usar semilla controlada y datos sintéticos. No usar información personal real ni credenciales de producción.

- **Reloj y fechas:** controlar tiempo en las pruebas de reglas que dependen de `America/Bogota`, citas futuras y cancelaciones vencidas.

5.1 Criterios de inicio

1. Alcance, ambiente y responsable de la ejecución identificados.
2. Revisión del commit/build que se probará y de las migraciones asociadas.
3. Variables de entorno verificadas; `NODE_ENV=test` para la suite automatizada.
4. Confirmar el host, usuario y base `DB_NAME_TEST`; no basta comprobar el sufijo del nombre.
5. Base de pruebas desechable, respaldos cuando correspondan y datos no productivos.
6. Casos con pasos, datos de entrada y resultados esperados definidos antes de iniciar pruebas de aceptación/no funcionales.

5.2 Seguridad de la base de pruebas

`globalSetup.js` crea la base, aplica migraciones/semilla, y `globalTeardown.js` intenta eliminarla. `testDbConfig.js` exige que `NODE_ENV=test`, que `DB_NAME_TEST` tenga identificador seguro, termine en `_test` y sea diferente de `DB_NAME`. **Esta validación no demuestra que el host sea local ni que la base sea desechable.** Verificar `DB_HOST`, `DB_PORT`, credenciales y `DB_NAME_TEST` antes de correr pruebas fuera del Compose temporal. No apuntar nunca a producción ni a una base con datos que deban conservarse.

6. Criterios de aprobación y rechazo

Aprobación propuesta

- El caso produce el resultado esperado y conserva las invariantes de negocio y datos.
- Todas las pruebas obligatorias del nivel ejecutado terminan correctamente; fallos intermitentes no se cuentan como aprobación.
- No quedan defectos bloqueantes/altos abiertos en autenticación, autorización, integridad de citas, exposición de datos o pérdida de información.
- El resultado registra commit, ambiente, fecha, versión, datos, pasos, evidencia y estado.
- Para aceptación de negocio, un responsable designado confirma explícitamente los criterios; esa aprobación no se presume por pasar tests técnicos.

Rechazo y severidad sugerida

Severidad	Ejemplos	Acción
Crítica	Acceso no autorizado a datos clínicos, exposición de secretos, pérdida/corrupción de datos o reservas incompatibles	Rechazar release; corregir y repetir regresión relacionada.
Alta	Flujo principal bloqueado, permisos incorrectos, cita/estado clínico mal registrado	Rechazar alcance afectado; corregir antes de aceptación.
Media	Función secundaria incorrecta, error de validación o UI con alternativa operativa limitada	Registrar y acordar corrección/prioridad antes de release.
Baja	Inconsistencia visual/textual sin impacto en seguridad/datos ni bloqueo funcional	Documentar, priorizar y resolver según criterio del responsable del producto.

Estos niveles son una propuesta de este plan y deben confirmarse con el equipo; no sustituyen una política institucional ya aprobada.

7. Estrategia y orden recomendado

1. **Preparación:** revisar cambios, actualizar casos y confirmar la base aislada.
2. **Unitarias:** validadores, fechas, permisos, servicios, utilidades y modelos.
3. **Integración:** API/Express con PostgreSQL temporal; verificar datos, estados y transacciones.
4. **Smoke:** salud API/BD y caminos esenciales tras build; agregar frontend/Nginx cuando se implemente E2E.
5. **Regresión funcional:** ejecutar los casos de rutas, roles, validación, mascotas, disponibilidad, citas e historias relacionados con el cambio.
6. **Seguridad:** ampliar matriz de autorización y análisis de configuración; mantener pruebas de inyección/hash/JWT.
7. **Sistema y aceptación:** ejecución manual/automatizada de flujos completos con usuarios sintéticos y criterios aprobados.

8. **No funcionales:** usabilidad, compatibilidad y luego rendimiento/carga/estrés en ambientes aprobados.
9. **Apoyo de IA, si se adopta:** usarla para sugerir casos o revisar resultados; validar manualmente casos y mantener oráculos deterministas.

8. Responsabilidades propuestas

Rol	Responsabilidad
Desarrollo backend	Mantener pruebas unitarias/integración, reglas, fixtures y migraciones de test.
Desarrollo frontend	Definir e incorporar pruebas de componentes, rutas y estados de interfaz; no declarar automatización hasta implementarla.
QA / responsable de pruebas	Mantener matriz de casos, evidencia, defectos, regresión y reporte de ejecución.
Representante del negocio/usuarios	Acordar criterios, participar en usabilidad/aceptación y aprobar resultados de negocio.
Responsable de ambiente/base	Confirmar configuración aislada, permisos, limpieza y protección de datos.

Los nombres de personas no se asignan aquí porque no fueron proporcionados.

9. Evidencia disponible y limitaciones

La bitácora de validación de formularios informa una ejecución histórica de `npm test` con **23 archivos y 334 pruebas aprobadas**, build de frontend y comprobaciones manuales de formulario. Las suites de backend actuales se encuentran en `back/tests/unit/` y `back/tests/integration/`. Estos datos son referencias documentales fechadas; no se ejecutó la suite ni se confirmó la misma cifra durante la creación de este plan.

No se encontraron suites automatizadas frontend/E2E, pruebas formales de carga/estrés/rendimiento, matriz de compatibilidad, plan de usabilidad con participantes ni acta de aceptación. La ausencia en el repositorio revisado no demuestra que nadie haya hecho pruebas informales fuera de él.